

EKI on 8 GB VRAM: Implementation Roadmap

Oct 7, 2026 · @Siddhant

Purpose and scope

The goal is a working, scaled-down reproduction of EKI (Expert Knowledge Internalization, arXiv 2609.36145) in a single notebook on one 8 GB GPU: train the DINOv2 expert, run Stage 1 (Precise Spatial Focus) and Stage 2 (FGRA), then run box-output inference and the paper's pixel-level IoU/F1 evaluation.

The paper's own setup (Qwen2.5-VL-7B, bf16 LoRA $r=32$, batch 8, 10 epochs on 120k images) does not fit 8 GB or a reasonable time budget. So the plan reproduces the *method* faithfully and scales the *budget*: a smaller or 4-bit base model, gradient accumulation instead of batch 8, and a training subset. Absolute numbers will be lower than Table II; the target is the paper's *relative* ordering: Baseline < Stage 1 only < FGRA only < full EKI.

Out of scope: the competitor re-trainings in Tables II–III, the ER (External Reliance) paradigm except as an optional extra, and the three-model comparison in Table VII.

What must be built

Six components come out of the paper; each maps to one notebook section.

Component	Paper reference	What it does	Trainable parts
Box targets	Sec. III-A	Mask $\rightarrow$ per-region min enclosing rectangle $[x1,y1,x2,y2]$; output text "Tampered. [boxes]" or "Real."	none
Forensic expert	Sec. IV-C1, V-A2	DINOv2-ViT-L + Mask2Former, BCE + Dice, trained on the same training set; only the backbone is kept	expert backbone + head

Component	Paper reference	What it does	Trainable parts
Stage 1, Text-Focused (TF)	Sec. IV-B1	PaddleOCR boxes with conf > 0.9 become prompts: "Is the text 't' located at b_ocr real or tampered?" → "Real." or "Tampered. b_gt"	vision encoder + projector (LLM frozen)
Stage 1, Image-Focused (IF)	Sec. IV-B2	Train a mining model M_pre, mark training images with IoU < 0.7 as hard, add magnified crops with "Is this image patch real or tampered?"	same as TF; loss = L_local + L_global
Stage 2, FGRA	Sec. IV-C2-3	Project LLM layer-1 visual hidden states with a head ϕ to the expert's dimension; loss = -mean cosine over K patch pairs; total = L_LM + L_FGRA	LLM (LoRA) + projector + ϕ ; vision encoder frozen
Inference + eval	Sec. V-A3, V-A5	Plain prompt, greedy decoding, parse boxes, rasterise to masks, per-image pixel IoU and F1, averaged over the test set	none

The FGRA loss, as written in the paper (Eq. 3):

$$\mathcal{L}_{\text{FGRA}} = -\frac{1}{K} \sum_{i=1}^K \cos(\phi(h^{(1)}_i, f_{\text{exp},i})), \quad l = 1$$

At inference neither OCR nor the expert is used; ϕ is discarded too.

Fitting into 8 GB

Use **Qwen2.5-VL-3B-Instruct with a 4-bit (NF4) LLM** as the primary base; treat the paper's 7B as a stretch goal. Same model family keeps the paper's architecture, token grid and prompt format, so every step transfers to 7B later unchanged.

Approximate training footprints (estimates from parameter counts; measure with `torch.cuda.max_memory_allocated` in Phase 0):

Configuration	Weights resident	Training peak (batch 1, grad checkpointing, ~400 tokens)	Verdict
3B, LLM NF4, ViT bf16	~3.5 GB	~5–6.5 GB	Primary path
3B, everything bf16 (paper precision)	~7.5 GB	> 8 GB	Does not fit
7B, LLM NF4, ViT bf16	~7 GB (untied embedding and lm_head stay ~2.2 GB in bf16)	> 8 GB	Training not feasible without CPU offload
7B, NF4, inference only	~7 GB	~7.5 GB	Possible for evaluating a 7B adapter trained elsewhere
DINOv2-ViT-L expert, last 12 blocks trained	~0.6 GB + optimizer state	~5–7 GB at 504 px, batch 2	Fits with checkpointing

The memory levers, in order of impact:

1. 4-bit LLM via bitsandbytes (QLoRA). The LLM is frozen in Stage 1 and LoRA-adapted in Stage 2, so quantising it costs little.
2. Gradient checkpointing on the LLM and the ViT; `use_cache=False` during training.
3. Batch 1 with gradient accumulation 8, which matches the paper's effective batch of 8.
4. 8-bit paged AdamW for LoRA, projector and ϕ parameters.
5. Precompute everything that is fixed: OCR results, box targets, hard-sample list, and the expert's Stage 2 features (cached to disk), so the expert never sits in VRAM next to the MLLM.
6. `attn_implementation="sdpa"`; Flash-Attention 2 only if the card is Ampere or newer and it installs cleanly.

Deviations from the paper, to be stated in any write-up: NF4 instead of bf16 LLM weights; 3B instead of 7B; LoRA on the vision encoder in Stage 1 (the paper says only that it uses LoRA $r=32$, $\alpha=64$); a training subset and fewer epochs; a lighter segmentation head for the expert (justified because the head is discarded before Stage 2).

Notebook layout and environment

One notebook, organised so each stage can be re-run alone from its checkpoint after a kernel restart. A single `CFG` dict at the top holds every path, resolution, threshold and hyperparameter.

Section	Contents	Writes to disk
0. Setup	Installs, GPU check (bf16 support, free VRAM), seeds, <code>CFG</code>	—
1. Data	Dataset readers (DocTamer is LMDB), resize to 504×504, mask → boxes, train subset selection	<code>boxes.jsonl</code> , <code>subset_ids.txt</code>
2. Expert	DINOv2-ViT-L + light head, BCE + Dice training, quick mask IoU check, feature caching	<code>expert_backbone.pt</code> , <code>feats/{id}.npy</code>
3. OCR + TF data	PaddleOCR pass, conf > 0.9 filter, TF instruction pairs	<code>ocr.jsonl</code> , <code>tf_pairs.jsonl</code>
4. M_pre + hard mining	Short baseline fine-tune, eval on train subset, IoU < 0.7 list, patch crops	<code>hard_ids.txt</code> , <code>patches/</code>
5. Stage 1	TF + IF + global localisation, ViT LoRA + projector	<code>stage1/</code> (merged ViT, projector)
6. Stage 2	LLM LoRA + projector + ϕ , L_LM + L_FGRA	<code>stage2/</code> (LoRA adapter, projector)
7. Inference	Plain prompt, greedy decode, box parser	<code>preds_{testset}.jsonl</code>
8. Evaluation + ablations	Pixel IoU/F1, per-dataset table, layer sweep, visualisations	<code>results.csv</code> , figures

Environment: Python 3.10+, PyTorch 2.x with CUDA, `transformers` recent enough for Qwen2.5-VL, `peft`, `bitsandbytes`, `accelerate`, `qwen-vl-utils`, `paddleocr` (run in a separate step or env if it conflicts with PyTorch's CUDA build), `opencv-python`, `scikit-image` for connected components, `lmdb`. Pin all versions in the first cell. Note the GPU generation: Turing cards (RTX 20xx) have no bf16, so compute dtype falls back to fp16 with loss scaling.

Roadmap by phase

Nine phases, run in order on one GPU; each ends with a gate that must pass before the next starts. Phases 2, 3 and 4 do not depend on each other and can be reordered.

1. Phase 0 — Environment and memory probe.

- Load Qwen2.5-VL-3B with NF4 LLM and bf16 ViT; run one forward + backward on a 504×504 image with a ~30-token answer, gradient checkpointing on.
- Confirm the visual token count is 324 (36×36 patches of 14 px, merged 2×2) and record the peak VRAM.
- Probe both training configurations: Stage 1 (ViT LoRA trainable, gradients through the frozen NF4 LLM) and Stage 2 (LLM LoRA, ViT frozen, all hidden states returned). Gate: both peaks < 7.2 GB, leaving headroom for fragmentation.

2. Phase 1 — Data and box targets.

- Obtain DocTamper (training, testing, FCD, SCD) first; FSTS, SACP and RIFLC are needed only for cross-domain columns.
- Resize image and mask to 504×504 together (nearest-neighbour for masks); all coordinates live in this 504 frame.
- Mask → connected components → min enclosing rectangles; drop components under a small area (for example 4 px) as label noise; serialise as "Tampered. [x1, y1, x2, y2], [...]" sorted top-left to bottom-right, or "Real." for an empty mask.
- Pick a stratified training subset (start at 10–20k images) and a fixed 1–2k validation split from the training set.
- Gate: overlay 50 random boxes on images and inspect; rasterised boxes vs. original mask IoU should be high on average.

3. Phase 2 — Forensic expert.

- DINOv2-ViT-L/14 backbone (`facebook/dinov2-large`), input 504×504 → 36×36 patch grid; light segmentation head (a few conv + upsample layers) in place of Mask2Former.

- Freeze the first half of the blocks, train the rest plus the head with BCE + Dice, batch 2 with accumulation, gradient checkpointing.
- Cache Stage 2 targets: last-layer patch features (1296×1024) $\rightarrow$ 2×2 average pool $\rightarrow$ 324×1024 , fp16, one file per image (~660 KB; ~13 GB for 20k images).
- Gate: expert box-level IoU (same metric as Phase 8, boxes taken from its masks) on the validation split is recorded; after Phase 4 it must be clearly above the Baseline, otherwise the teacher has nothing to teach.

4. Phase 3 — OCR and Text-Focused data.

- Run PaddleOCR over the training subset (at 504 or original resolution, then rescale boxes into the 504 frame); keep instances with $\text{conf} > 0.9$.
- For each sampled OCR box: if it overlaps the tampered mask, target is "Tampered. b_gt" where b_gt is the box of the mask *inside* that OCR region; else "Real."
- Sample a fixed number of queries per image (for example 2–4) with a balanced real/tampered ratio.
- Gate: visual spot-check of 30 pairs; tampered fraction is not extreme (aim 30–50%).

5. Phase 4 — M_pre and hard-sample mining.

- Fine-tune the baseline on global localisation only (Stage 1 settings, short: 1 epoch on the subset). This run doubles as the "Baseline" row of the ablation.
- Evaluate on the training subset; images whose pixel IoU between the union of predicted boxes and the union of GT boxes is < 0.7 are hard (no prediction = IoU 0). Because M_pre has seen these images, it under-reports hardness; if few samples come out hard, train M_pre on one half and mine the other, then swap.
- For each hard image crop every tampered region with context padding plus an equal number of random authentic crops, resize to 504×504 , label "Tampered" or "Real".
- Gate: hard fraction reported; crops inspected.

6. Phase 5 — Stage 1 training.

- Trainable: LoRA on ViT attention + MLP, full projector (merger). LLM frozen in NF4 but still in the backward path, since the loss is L_{LM} . PEFT's `prepare_model_for_kbit_training` freezes every parameter, so set `requires_grad` on the ViT LoRA and projector after calling it, and print the trainable-parameter count to confirm.
- Data: shuffled mix of global localisation, TF pairs and IF crops; loss on answer tokens only (prompt tokens masked to -100).
- After training, merge the ViT LoRA into the bf16 ViT weights and save ViT + projector.

- Gate: validation box IoU beats M_{pre} .

7. Phase 6 — Stage 2 training with FGRA.

- Load the merged Stage 1 ViT (frozen) and projector (trainable); add LoRA $r=32$, $\alpha=64$ to the LLM; add $\phi = \text{Linear}(2048 \rightarrow 1024)$ or a 2-layer MLP.
- Call the model with `output_hidden_states=True` and take `hidden_states[1]` (the first decoder block's output). Avoid forward hooks: with gradient checkpointing they fire again during recomputation. Select the 324 positions where `input_ids == image_token_id`; FGRA loss against the cached features; total = $L_{LM} + L_{FGRA}$.
- Log both losses separately; cosine similarity should rise steadily.
- Gate: validation IoU beats Stage 1; FGRA-only run (Stage 2 from the original ViT) also beats Baseline.

8. Phase 7 — Inference.

- Prompt "Detect and localize the tampered text in this image", greedy decoding, `max_new_tokens` sized for the maximum box count seen in training.
- Robust parser: regex for integer quadruples, clip to $[0, 504]$, fix $x_1 > x_2$, treat unparseable output as "Real." and count those cases.
- Gate: parse failure rate under 1%.

9. Phase 8 — Evaluation and ablations.

- Rasterise predicted and GT boxes to binary masks, pixel IoU and F1 per image, mean over the test set, per dataset.
- Ablation rows to reproduce Table V's shape: Baseline, TF, IF, TF+IF, FGRA only, full EKI. Layer sweep $l \in \{1, 4, 14, \text{last}\}$ for Fig. 6. PCA visualisation of layer-1 visual tokens for Fig. 7.
- Gate: the ordering Baseline < TF+IF < FGRA only < full EKI holds on DocTamper-TestingSet.

Gaps in the paper and the choices made

The paper leaves several implementation details open; each is fixed here and should be logged in `CFG` so it can be varied later.

Open point	What the paper says	Decision for this build
Input resolution	"Resized to 512×512"	504×504, the nearest size Qwen2.5-VL processes without its own resize (multiple of 28); also gives DINOv2 a clean 36×36 grid
Making $N = M$	Grids "unified in preprocessing"	Both at 504; DINOv2's 36×36 pooled 2×2 to 18×18 = 324, matching Qwen's merged tokens
Which expert layer	"Feature embeddings" of the backbone	Last-layer patch tokens (after final norm), CLS and register tokens dropped
Layer " $l = 1$ "	First LLM layer	Output of the first decoder block (<code>hidden_states[1]</code>); <code>hidden_states[0]</code> (embeddings) tried as a variant
Projection head ϕ	"Learnable projection head"	2-layer MLP with GELU; linear as a variant
FGRA loss weight	Plain sum, Eq. 4	Weight 1.0, matching the paper; tune only if L_{FGRA} dominates
Where LoRA goes in Stage 1	Not stated	ViT q/k/v/o + MLP; projector fully trained
Multiple tampered regions	Not stated	One box per connected component, small ones dropped; option to merge boxes closer than a few px
IF crop size	Not stated	Tampered box padded to $\geq 3 \times$ its size and ≥ 64 px, random authentic crops of matching size
Mixing ratio of TF / IF / global	Not stated	Equal sampling weight per source; adjust if one dominates

Open point	What the paper says	Decision for this build
Authentic images and metrics	Not stated	Empty GT and empty prediction counts as $\text{IoU} = \text{F1} = 1$; any false box on an empty GT counts as 0; report both with and without authentic images
Expert training data	Same set as the MLLM	Same training subset, so no leakage into test sets
Coordinate format	$[x1, y1, x2, y2]$	Absolute integer pixels in the 504 frame, matching Qwen2.5-VL's grounding convention

One more open point: the paper does not say which data Stage 2 trains on. This build uses global localisation samples only, so expert features are needed just for full 504×504 images. If TF or IF samples are added to Stage 2 later, the IF crops need their own cached expert features, and TF prompts add text tokens but leave the 324 visual positions unchanged.

Compute and time budget

One full EKI run is roughly 40–50 GPU-hours on an RTX 3070/4060-class card, assuming ~1.2 s per training sample (3B, NF4, checkpointing, ~400 tokens). These are planning estimates; replace the per-sample time with the Phase 0 measurement and rescale.

Step	Samples $\times$ epochs	Est. hours
Expert training (DINOv2-L, half frozen)	$15\text{k} \times 3$	4–6
Feature caching + OCR pass	15k	2–3
M_pre / Baseline (global only)	$15\text{k} \times 1$	~5
Hard-sample mining (inference on train subset)	15k	3–4
Stage 1 (global + TF + IF, capped at ~30k samples)	$30\text{k} \times 1$	~10
Stage 2 (global, with FGRA)	$15\text{k} \times 2$	~10
Evaluation on 2k-image samples of each test set	$7 \times 2\text{k}$	3–4

Step	Samples × epochs	Est. hours
Full DocTammer-TestingSet (30k) at the end	30k	6–8

Keep the cost down by iterating on a 2k-image slice of each test set and running full test sets only for the final table. Ablation rows (TF only, IF only, FGRA only) each cost a training run; run them on a 5k subset rather than the full 15k. Save a checkpoint every ~500 optimiser steps so a crashed or timed-out session resumes.

Risks and fallbacks

Risk	Sign	Fallback
Out-of-memory in Stage 1 (backward through frozen 4-bit LLM into the ViT)	OOM on long answers or many boxes	Cap answer length; shorten prompts; drop to 448×448 (16×16 tokens, DINOv2 at 448 pooled 2×2); train ViT with LoRA on later blocks only
Visual token order differs from DINOv2's raster order (Qwen2.5-VL uses window attention inside the ViT)	FGRA cosine plateaus near 0	Check the merger output is row-major: align a test image's tokens with a shifted copy and confirm the shift appears in the right positions
Expert is too weak to teach	Expert val IoU near the baseline	Train more blocks or epochs; fall back to DINOv2-ViT-B with full fine-tuning
QLoRA accuracy loss vs bf16	Baseline far below paper's 0.696 Doc-Test IoU even after scaling	Accept; compare only relative gains. If a larger GPU becomes available, rerun bf16

Risk	Sign	Fallback
Datasets unavailable	Download or licence blocked (DocTamper is distributed on request; FSTS is a 2025 preprint; SACP and RIFLC need Tianchi accounts)	Start with DocTamper alone and report in-domain + FCD/SCD only
Model outputs malformed boxes or hallucinates many boxes	High parse-failure rate, low precision	Consistent target formatting, cap box count, inspect M_pre outputs early
PaddleOCR conflicts with PyTorch's CUDA	Import errors	Run OCR in a separate CPU environment or Colab session; it is a one-off preprocessing step
Session time limits (Colab/Kaggle)	Runs killed mid-epoch	Step-level checkpoints with optimiser and dataloader state; resume cell
FGRA loss swamps L_LM	L_LM stops falling	Lower the FGRA weight to 0.5 or 0.1 and note the change

Review of this roadmap

A full read-through found six errors or omissions, now fixed above, and nine shortfalls that remain open and should be decided before or during Phase 0.

Fixed during review

- Phase 2's gate compared the expert with the Baseline from Phase 4, which had not run yet. It now records the expert's score and checks it against the Baseline after Phase 4.
- Phase 0 probed only one configuration. Stage 1 and Stage 2 have different memory profiles (gradients into the ViT vs. LoRA in the LLM), so both are now probed.
- Hard mining used "box IoU" without defining it for multiple or missing boxes. It now uses pixel IoU of box unions, with a held-out-half option because M_pre has seen its own training images.

- Phase 6 used a forward hook to read layer 1. Under gradient checkpointing hooks fire again on recomputation; it now uses `output_hidden_states`, which costs only ~60 MB at this sequence length.
- Stage 1 did not mention that PEFT's k-bit preparation freezes all parameters, which would silently leave the ViT untrained. Added an explicit re-enable and parameter-count check. The Stage 2 data mix, also missing, is now stated under the gaps table.

Still open

- *Unverified estimates.* Memory and hour figures are derived from parameter counts, not measured. Phase 0 must replace them before committing to subset sizes.
- *Too few authentic examples.* DocTamper's training images are, to my understanding, all tampered, so "Real." targets come only from TF queries and IF crops. The model may learn to always emit a box, which hurts on FSTS and authentic images. Check the real/tampered ratio in Phase 1 and add authentic crops or FSTS authentic images if it is skewed.
- *Underfitting.* The paper trains 10 epochs; this plan uses 1–2. Stage 2's LoRA in particular may not converge. Select checkpoints by validation IoU and extend epochs on the smaller subset if curves are still falling.
- *Noise in ablations.* Single-seed runs on 5k subsets may produce gaps smaller than run-to-run variance. Run the key rows (Baseline, FGRA only, full EKI) with 2–3 seeds or report bootstrap confidence intervals.
- *Quantisation confound.* FGRA's gain on an NF4 base is untested; a smaller or larger gain than the paper's could come from quantisation rather than the method.
- *Resolution limit.* Square 504×504 resizing, inherited from the paper, distorts aspect ratio and shrinks small characters on large scans. Out of scope to change, but expect weaker results on high-resolution sets.
- *No augmentation in Stage 2.* Cached expert features fix the input, so random crops, JPEG or flips are impossible in Stage 2 unless the expert runs on the fly (~0.6 GB fp16, which still fits).
- *Inference latency not measured.* Table IV's claim (EKI ≈ vanilla speed) needs a timing cell in Phase 7: mean ms per image for Baseline and EKI on the same GPU.
- *Dataset access.* Licences and download routes for DocTamper, FSTS, SACP and RIFLC are not yet confirmed; this gates Phase 1 and should be checked first.